FAST-NUCES

National University of Computer and Emerging Sciences
Peshawar Campus

Computer Organization and Assembly Language Lab

LAB # 04

Loop and Conditions

Student Name	Waleeja Khan	**Roll No.**	25P-0005
Section	BAI-3A	**Date**	
Instructor	Hamad Ahmad	**Marks**	____ / 20

Learning Objectives

- Understand and apply JMP, conditional jumps, and loops in 8086 assembly.
- Access and process elements of an array using registers and memory.
- Use conditions to control program flow.
- Trace changes in registers, memory, and flags during execution.
- Predict program behavior and final values before execution.
- Verify program execution using the debugger.

Instructions

1. Complete all tasks using DOSBox ,Nasm.
2. Use of AI-generated answers or solutions from classmates is strictly prohibited.
3. Use the debugger for instruction-by-instruction investigation where requested.
4. Answers and explanations should be written in the student's own words.
5. Save the completed document as: Lab1_<RollNo>_<Name>.docx

Task 1. Count 3:

Given:

arr db 3, 7, 3, 9, 3, 2, 7, 3

count how many times 3 appears. Store the result in AX

Code:

```
[org 0x0100]

MOV SI, 0
MOV BL, 7
MOV CL, 0
label:
MOV AL, [arr+SI]
CMP AL, 3
JNZ Next
ADD CX, 1
Next:
ADD SI, 1
SUB BL, 1
jnz label
MOV AX, 4c00h
INT 21h

arr: db 3, 7, 3, 9, 2, 7, 3
```

Task 2 Reverse the Array :

Given:

arr db 10, 20, 30, 40, 50, 60

Reverse the array **in-place**:

60 50 40 30 20 10

Restriction: Do not create a second array.

Twist: The number of loop iterations should not be equal to the number of array elements.

Code:

```
[org 0x0100]

MOV SI, 0
MOV DI, 5
MOV CX, 3

label:
MOV AL, [arr+SI]
MOV BL, [arr+DI]
MOV [arr+SI], BL    ; store back swapped
MOV [arr+DI], AL
ADD SI, 1
ADD DI, 1
SUB CX, 1
JNZ label

MOV AX, 4c00h
INT 21h

arr: db 10, 20, 30, 40, 50, 60
```

Task 3 . The Carry Flag challenge :

Add all elements of:
arr db 200, 100, 50, 25

and determine whether the sum exceeds **255**. If it is then store 1 in AL otherwise 0

Twist: The individual elements fit inside 8 bits, but the **sum does not**.

Code:

```
[org 0x0100]

MOV SI, 0
MOV CX, 4
MOV Bl, 0
MOV DL, 0

label:
ADD Bl, [arr+SI]
JNC noCarry
MOV dl, 1

noCarry:
MOV SI, 1
loop label
MOV Ah, 4ch
MOV AL, DL
INT 21H
arr: db 200, 100, 50, 25
```

Task 4 :

How many times the loop L1 executed and how many times the CX executed.

```
MOV CX, 3
L1:
   DEC CX
   JZ  L2
   LOOP L1
L2:
```

Answer: Loop L1 executes for 1 time, while cx executes for 2 times.